

06 - নিজató끼 팀 - 최종 보고서

(1) Team Info (팀 정보)

1.1. 과제명

전통주 소비 입문자를 위한 AI 기반 전통주 큐레이션 서비스

1.2. 팀 정보 (팀 번호, 팀 이름 등)

06 নিজató끼 팀

1.3. 팀 구성원

2203076 육란 - 팀장, AI, 백엔드 / 2171022 송연우 - 팀원, 백엔드 / 2171027 안유경 - 팀원, 프론트엔드

(2) Project Summary

2.1. 문제 정의

2.1.1. 과제의 배경과 필요성

통계청 및 농림축산식품부 자료「전통주 산업 활성화 대책」에 따르면, 국내 전통주 시장 규모는 최근 10여 년간 2010년 433억 원에서 2022년 1,629억 원으로 약 4배 성장하며 빠른 성장세를 보이고 있다. 이는 전통주가 단순한 지역 특산품을 넘어 새로운 소비 트렌드로 자리 잡고 있음을 보여준다.

그러나 이러한 성장에도 불구하고, 전통주 입문자들이 전통주를 쉽게 접하고 즐길 수 있는 환경이 여전히 부족하다. 많은 소비자들이 전통주에 관심을 가지지만 실제 구매나 음용 단계에서는 “어떤 술을 선택해야 할지 모르겠다”는 이유로 망설이는 경우가 많다. 이는 전통주 관련 정보가 여러 경로에 흩어져있고, 개인의 취향을 반영한 맞춤형 추천이나 상황별 가이드가 부족하기 때문이다.

본 과제의 사전 설문조사(20~30대 전통주 관심자 15명, 복수 응답 허용)에 따르면, 전통주 구매를 망설이는 가장 큰 이유로 ‘평소 마시던 술과 비슷한 전통주를 알지 못한다’(73.3%), ‘가격대나 도수 외의 정보 부족’(66.7%)을 꼽았다. 또한 전통주 선택 과정에서 ‘내 취향에 맞는 기준을 모르겠다’(80%), ‘어떤 음식과 함께 마셔야 할지 모르겠다’(53.3%)라는 응답이 다수였다. 전통주 정보를 얻는 경로 역시 소셜미디어(55.6%), 지인 추천(33.3%), 인터넷 검색(11.1%)에 의존하고 있어 정보의 체계성과 접근성이 부족한 실태를 알 수 있었다.

따라서, 전통주 입문자들에게는 필요한 정보를 한눈에 얻고, 자신의 취향과 상황에 맞게 전통주를 즐길 수 있도록 지원하는 종합 큐레이션 서비스가 필요하다.

2.1.2. 목표 사용자 (target customer) 설정 및 이유

본 과제는 전통주 소비에 입문하고자 하는 20~30대 젊은 층을 주요 목표 사용자로 설정한다.

이와 같이 설정한 이유는, 최근 10여년간 20대와 30대의 전통주 소비가 각각 63%, 78% 증가하며 주요 소비층으로 떠오르고 있기 때문이다. (출처: <https://www.mk.co.kr/news/business/10197004>) 또한, 서울국제주류페어 참석자의 50% 이상이 20-30대로 구성되었으며, 그 중 20% 전통주에 대한 직접적인 관심을 보였다.

이를 기반으로, 개성 있는 경험 소비, 로컬 브랜드 가치 등을 중시하는 젊은 20-30대의 MZ세대는 SNS, 숏폼 콘텐츠, 지역 체험형 마케팅 등을 통해 전통주를 새로운 문화적 콘텐츠 요소로서 인식하고 있다는 것을 파악할 수 있다. 즉, 과거 특정 세대의 전유물로 여겨지던 전통주가 젊은 세대가 새롭게 즐기고 향유하는 문화 콘텐츠로 자리잡고 있음을 보여준다.

따라서, 본 과제는 전통주를 처음 접하는 젊은 세대를 대상으로 취향 분석 기반의 맞춤형 전통주 추천을 중심으로 한 큐레이션을 제공함으로써, 전통주의 접근성과 친밀도를 높이고자 한다.

2.1.3. 본 과제를 통해 완화하고자 하는 문제 정의(pain points)

전통주 소비에 입문하고자 하는 20-30대들은 다음과 같은 문제를 경험하고 있다.

1) 정보의 비대칭성으로 인한 높은 진입장벽

전통주는 맥주나 와인과 달리 체계화된 정보가 부족해 입문자가 선택 기준을 세우기 어렵다. 이로 인해, 많은 입문자들이 기존에 소비하던 주류 취향과 전통주를 연결하는 데 어려움을 겪으며, “어디서부터 시작해야 할지 모르겠다”며 망설이는 경우가 많다.

2) 개인의 세부 취향 패턴을 반영한 전통주 추천 서비스의 부재

기존의 전통주 추천 서비스는 단맛, 신맛, 청량감 등의 단편적인 주류 특성에 의존하고, 사용자의 세부적인 취향 패턴을 충분히 반영하지 못해 단발성 추천에 그친다. 그 결과, 추천의 만족도가 떨어지고, 지속적인 전통주 소비로 이어지는데 한계가 있다.

3) 전통주 음용문화 및 구체적 상황별 가이드라인 부족

전통주는 계절, 날씨, 음식, 상황과의 조화를 통해 그 가치를 온전히 체감할 수 있는 주류임에도, 이를 안내하는 체계적인 플랫폼이 부족하다. 입문자들은 “언제, 무엇과 함께 마셔야 하는가”라는 질문에 대한 명확한 답을 얻기 어렵다.

4) 전통주 정보의 파편화로 인한 낮은 정보 접근성, 검색 편의성

전통주 정보는 생산자·유통사·리뷰 플랫폼 등 다양한 출처에 흩어져 있어 상세 정보를 확인하기 위해서는 여러 경로를 탐색해야 하는 번거로움이 존재한다. 또한 출처별로 제공되는 정보 수준이 달라, 원하는 정보를 얻기까지 상당한 시간과 노력이 소요되어, 사용자 경험과 접근 효율성이 저하되는 문제점이 있다.

2.2. 기존연구와의 비교

2.2.1. 유사 서비스 분석

1) 와인 구독 서비스 ‘퍼플독’

서비스 형태	“내 와인 스타일 찾아보기” 취향 테스트
장점	1. 와인의 종류를 세분화하여 취향을 파악할 수 있도록 질문을 나누고 있다. 2. 와인의 도수, 산미, 당도 등을 선택할 때, 수치가 아닌 ‘느껴진다’, ‘덜 느껴진다’로 구성해, 쉽게 선택할 수 있다.

단점	1. 사용자 맞춤형 와인을 추천해주지 않아, 서비스 구독자가 아닌 경우엔 직접 검색을 통해 매핑해야 한다. 2. 상황별, 음식 페어링 정보를 제공하지 않는다. 3. 와인을 마셔보지 않았다면 취향을 파악하기 힘들다.
----	---

서비스 형태	AI가 사용자의 취향에 맞는 와인을 정기구독 형태로 제공
장점	사용자의 설문과 정기 구독 피드백을 받아 지속적으로 취향을 반영해 와인을 추천한다.
단점	1. 약 39,000원 ~ 300,000 이상의 가격대의 정기 구독 서비스를 결제해야 추천 정보를 받아볼 수 있다. 2. 추천 받을 수 있는 와인 종류가 보통 1~2개로 제한되어 있다. 3. 사용자가 직접 제공하는 데이터에만 의존하여 취향을 업데이트 하고 있다.

=> 와인 구독 서비스인 '퍼플독'은 와인 특성을 세분화하여 설문을 진행하고 구독 피드백을 반영하는 장점이 있으나, 특정 가격대 이상의 구독 서비스에 의존해야 하고, 추천의 다양성이 제한적이다. 또한 사용자가 제공한 데이터만을 기반으로 하기 때문에 취향 업데이트가 정교하지 않을 수 있다.

2) 전통주 종합 정보 사이트 - '더술닷컴'

서비스 형태	"나만의 우리술 찾기" 인공지능 소몰리에 테스트
장점	사용자가 원하는 전통주를 취향에 맞게 추천 받을 수 있다.
단점	1. 사용자가 도수, 산미, 당도 등을 1~5 사이의 수치로 직접 선택해야 한다.. 2. 입력한 수치에 해당하는 전통주를 매핑하는 단순 추천 로직. 3. 사용자에게 제공받는 정보가 제한적이며, 단발성 추천에 그치기 때문에 추천 결과의 만족도가 떨어질 수 있다.

=> 전통주 종합 정보 사이트인 '더술닷컴'의 "나만의 우리술 찾기"는 사용자가 직접 입력한 도수·산미·당도 등의 수치(1~5)를 기반으로 전통주를 매핑하는 단순 로직에 의존한다. 이는 전통주 경험이 부족한 입문자에게 적합하지 않으며, 단발성 추천에 머물러 사용자의 피드백이나 취향 변화를 반영하지 못한다.

2.2.2 본 과제가 갖는 차별성

앞서 살펴본 주류 추천 서비스들은 일정 부분 편의성을 제공하나, 전통주 입문자가 필요로 하는 종합적이고 지속적인 맞춤형 경험을 제공하지는 못한다. 따라서 이러한 단점을 개선해, 다음과 같은 차별화 포인트를 가지고자 한다.

기존 서비스의 한계	개선 및 차별화 포인트
단순 수치 매핑 / 평점 기반 추천	직, 간접적 데이터를 모두 반영한 가중치 기반 추천
사용자가 직접 입력하는 데이터에만 의존	초기 사용자 전통주 선호도 테스트를 포함하여 사용자의 누적 행동 패턴과 (전통주 클릭 패턴, 찜 목록, 평점) 날씨등의 물리적 정보를 모두 반영
정적 프로필에 따른 단발성 추천	사용자의 선호도를 반영한 피드백 루프를 통해 동적으로 프로필 업데이트 및 추천

2.3. 제안 내용

앞서 언급한 **pain point**와 기존 서비스의 한계를 해결하기 위해, 본 과제는 **AI** 기반의 **4단계** 학습형 전통주 큐레이션 시스템을 제안한다. 구체적인 기능은 다음과 같다.

기능 1) 진입장벽 제거를 위한 쉬운 주류 취향 테스트

어플 최초 진입시, 사용자는 온보딩 화면에서 주류 취향 테스트를 진행한다. 이때, 각 질문은 와인, 맥주 등 익숙한 기존 주류의 특성을 기반으로 한 간단한 이진선다형으로 구성되며, 결과는 맞춤형 추천의 초기 데이터로 활용된다. 테스트가 끝나면 사용자는 자신의 전통주 취향 프로필을 확인하고, 이를 바탕으로 추천된 전통주를 홈 화면에서 바로 볼 수 있다.

=> 예시 질문: 어떤 느낌의 목넘김을 선호하시나요?

=> 예시 선택지: **A.** 흑맥주, 레드 와인처럼 묵직하고 진한 맛 / **B.** 라거, 화이트와인처럼 가볍고 산뜻한 맛

해당 기능을 통해 사용자는 전통주 관련 지식이 부족해도, 기존에 즐겨 마시던 주류를 기반으로 취향을 쉽게 파악할 수 있고, 전통주 소비에 쉽게 입문할 수 있게 된다.

기능 2) 개인화된 맞춤형 전통주 추천 기능

① AWS Personalize 기반 추천

- 초기 취향 프로파일(직접 데이터)과 전통주 조회 기록, 평점, 찜 목록(간접 데이터)을 결합하여 개인화된 추천을 제공한다.

AWS Personalize 기반의 추천 엔진을 구축하고, 온보딩 화면에서 수집한 초기 주류 취향 프로파일로 사용자의 행동 패턴(사용자의 전통주 조회 기록, 평점, 찜 목록)에 따라 추천 가중치를 동적으로 반영한다.

각 가중치는 전통주 조회 기록을 **0.5**, 평점(리뷰)를 **0.5**, 찜 목록을 **1**로 한다.

- 초기 테스트 이후에는 사용자에게 추가적인 입력을 요구하지 않고, 사용자가 어플을 이용하기만 하면 취향 패턴을 분석해서 이를 추천에 반영하는 구조로서, 사용자 친화적인 경험을 제공하는 것을 목표로 한다.

② 기상청 초단기예보 API 기반 추천 기능
기상청 초단기예보 API를 통해 사용자의 위치를 기반으로 수집한 날씨 데이터(간접 데이터)를 활용하여, 특정 기온이나 날씨 조건에 따라 특정 주종이나 특성을 가진 전통주를 추천한다. 날씨별 추천 기준은 다음과 같다.

구분	조건	조정 기준	추천 이유
기온별	더운 날 (25도 초과)	청량감: +1 (최대 5), 도수: -3 (최소 3), 단맛: +1 (최대 5)	시원, 청량한 낮은 도수의 술
	추운 날 (10도 미만)	도수 +5 (최대 50), 단맛 +1 (최대 5), 청량감 -1 (최소 1)	몸을 따뜻하게 해줄 높은 도수의 술
	적당한 날씨 (10-25도)	기본 사용자 선호도 유지	-
날씨별	비 오는 날	추천 주종 = "탁주"	부침개와 잘 어울리는 탁주
	맑은 날	청량감 +2 (최대 5)	깔끔하고 청량감 있는 술

기능 3) Langchain 챗봇을 통한 사용자 상황인식 추천
Gemini 와 LangChain을 기반으로 하는 챗봇을 구축하여, 사용자가 입력한 텍스트(직접 데이터) 에서 감정·상황(간접적 데이터)을 인식하고, 맥락에 맞는 전통주와 음식 페어링 정보를 함께 제공한다.
사용자는 챗봇에게서 특정한 상황에 대한 전통주를 1:1로 직접 추천받을 수 있으며, 전통주 음용 가이드에 대한 정보를 함께 제공받아 좀 더 조화롭게 전통주를 즐길 수 있다.

기능 4) 쉬운 전통주 정보 접근을 위한 OCR 기능
사용자가 카메라로 전통주 라벨을 촬영하면 Naver Clova OCR을 통해 해당 텍스트를 인식한다. DB에 있는 전통주와 매칭에 성공하면, 해당 전통주의 상세 정보(이름, 주류 특성값, 어울리는 음식, 구성 원재료)를 사용자에게 제공하여, 외부 검색의 번거로움을 해소할 수 있다.

- 2.4. 기대 효과 및 의의
- 1) 전통주 소비 입문자의 진입장벽 완화
와인이나 맥주 등 익숙한 주류를 기반으로 한 간단한 취향 테스트를 통해, 전통주 경험이 없는 사용자도 손쉽게 자신의 취향을 확인할 수 있으며, 이는 “어디서부터 시작해야 할지 모르겠다”는 심리적 장벽을 낮추어 자연스럽게 전통주 소비로 유도할 수 있다.
 - 2) 정교한 개인화와 지속적 추천을 통한 사용자 경험 향상
별도의 추가 입력 없이도, 사용자가 어플을 이용하는 과정에서 축적되는 조회 기록, 평점, 찜 목록 등이 취향 패턴으로 반영되어 추천 결과가 지속적으로 정교화된다. 단발성이 아닌 동적 업데이트를 기반으로 한 장기적인 추천 구조를 통해 높은 사용자 만족도를 기대할 수 있다. 뿐만 아니라, 날씨, 상황, 음식과의 조합을 고려한 맞춤형 가이드와 전통주 관련 배경 지식 콘텐츠를 제공함으로써, 소비자는 문화적으로 풍부한 경험을 할 수 있다.
 - 3) 정보 접근성 향상 및 탐색 편의성 강화
OCR을 통해 전통주 라벨을 촬영해 전통주 상세 정보를 즉시 확인할 수 있다. 이는 사용자로 하여금 효율적이고 직관적인 정보 탐색이 가능하게 한다.
 - 4) 전통주 시장의 지속 가능한 성장 기반 마련
“어렵고 낯선 술”로 여겨지던 전통주를 “쉽게 즐길 수 있는 문화적 콘텐츠”로 재해석하여, 특히 20-30대의 MZ세대를 중심으로 전통주에 대한 관심을 실제 소비로 전환시켜 지속 가능한 시장 성장 기반을 마련할 수 있을 것으로 예상된다.

2.5. 주요 기능 리스트

ID	카테고리	메인 기능	세부 기능	설명
F-001	회원	회원가입	구글 소셜 회원가입	구글 계정으로 회원가입 가능
F-002			카카오 소셜 회원가입	카카오 계정으로 회원가입 가능
F-003		로그인	구글 로그인	구글 계정으로 로그인 가능
F-004			카카오 로그인	카카오 계정으로 로그인 가능
F-005		로그아웃	로그아웃	로그인한 계정에서 로그아웃
F-006		회원탈퇴	계정 삭제	사용자가 계정을 탈퇴할 수 있음
F-007		프로필 관리	내 프로필 수정	마이페이지에서 내 프로필(닉네임) 수정 가능
F-008	추천	취향 기반 추천	취향 테스트	처음 가입 시 선호도 테스트(2지선다)를 통해 단맛/쓴맛/신맛/바디감/탄산/도수 선호도를 DB에 저장

F-009			취향 테스트 결과 조회	마이페이지에서 회원가입 시 진행했던 전통주 선호도 테스트 결과를 조회 가능
F-010			개인화 추천	저장된 선호도, 사용자 행동 기록을 기반으로 맞춤 전통주 추천
F-011		날씨 기반 추천	위치 기반 날씨 조회	현재 위치를 기반으로 날씨 정보 표시
F-012			날씨별 추천	날씨 상황(맑음/비/기온)에 따라 전통주 추천
F-013		가격 기반 추천	가격 필터링 추천	가격 조건(예: 3만원 이하)에 맞춰 추천
F-014	검색	전통주 검색	전통주 종류 필터링	탁주/약주/증류주 등 종류별 필터링 가능
F-015			키워드 필터링	특정 키워드(예: 여름, 파티 등)로 검색 결과 제한
F-016	전통주 정보	상세 조회	전통주 상세페이지 조회	전통주의 상세 정보(이미지, 이름, 가격, 전통주 분류, 설명, 원재료, 키워드, 양조장 정보, 추천 조합 음식 등) 조회
F-017			사용자 선호도 반영	상세 페이지 조회 시 사용자 Interaction 기록 후 선호도 즉시 반영
F-018	찜	찜하기	전통주 찜 등록	특정 전통주를 찜 목록에 추가
F-019			전통주 찜 해제	찜 목록에서 제거 가능
F-020			사용자 선호도 반영	전통주를 찜할 시 사용자 Interaction 기록 후 선호도 즉시 반영
F-021		찜 목록	마이페이지 찜 목록 조회	사용자가 찜한 전통주 목록 확인
F-022	리뷰	리뷰 작성	평점 리뷰	전통주에 평점 남기기
F-023			사진 리뷰	사진을 첨부하여 리뷰 남기기(선택)
F-024			글 리뷰	제목/내용을 작성하여 리뷰 남기기
F-025			사용자 선호도 반영	사용자가 남긴 리뷰 평점에 기반해 Interaction 기록 후 선호도 즉시 반영
F-026		리뷰 조회	전통주 리뷰 목록	특정 전통주에 대한 모든 리뷰 조회
F-027			리뷰 평균 평점 표시	특정 전통주의 모든 리뷰에 대해 집계된 평균 평점 표시
F-028			내 리뷰 목록 조회	마이페이지에서 내가 작성한 리뷰 목록 확인
F-029		리뷰 관리	리뷰 수정	내가 작성한 리뷰 수정 가능
F-030			리뷰 삭제	내가 작성한 리뷰 삭제 가능
F-031	챗봇	전통주 Q&A	질문 응답	전통주 관련 질문에 대해 AI 답변 제공
F-032		대화 기록	대화 저장	사용자의 챗봇 대화 내용을 저장
F-033			이전 대화 불러오기	과거 챗봇 대화 이력 조회
F-034	OCR	전통주 라벨 인식	갤러리 업로드	갤러리에서 전통주 사진 업로드 후 인식
F-035			사진 촬영 업로드	카메라로 촬영한 전통주 사진 업로드 후 인식
F-036			텍스트 인식 결과	OCR로 추출된 정보를 Gemini가 분석하여 전통주 정보 제공(단맛, 신맛, 쓴맛, 도수, 어울리는 음식, 원재료 등)
F-037	축제	전통주 축제 조회	축제 목록	크롤링해온 축제 목록을 연도별로 조회
F-038	전통주 소개	전통주 이야기 보기	전통주 입문을 위한 이야기 제공	전통주(탁주, 약주, 증류주)를 만드는 과정, 술과 관련된 관용구나 관습 등 전통주와 관련된 이야기 제공

3) Project-Design (과제 설계)

3.1 요구사항 정의

3.1.1 사용자 특성

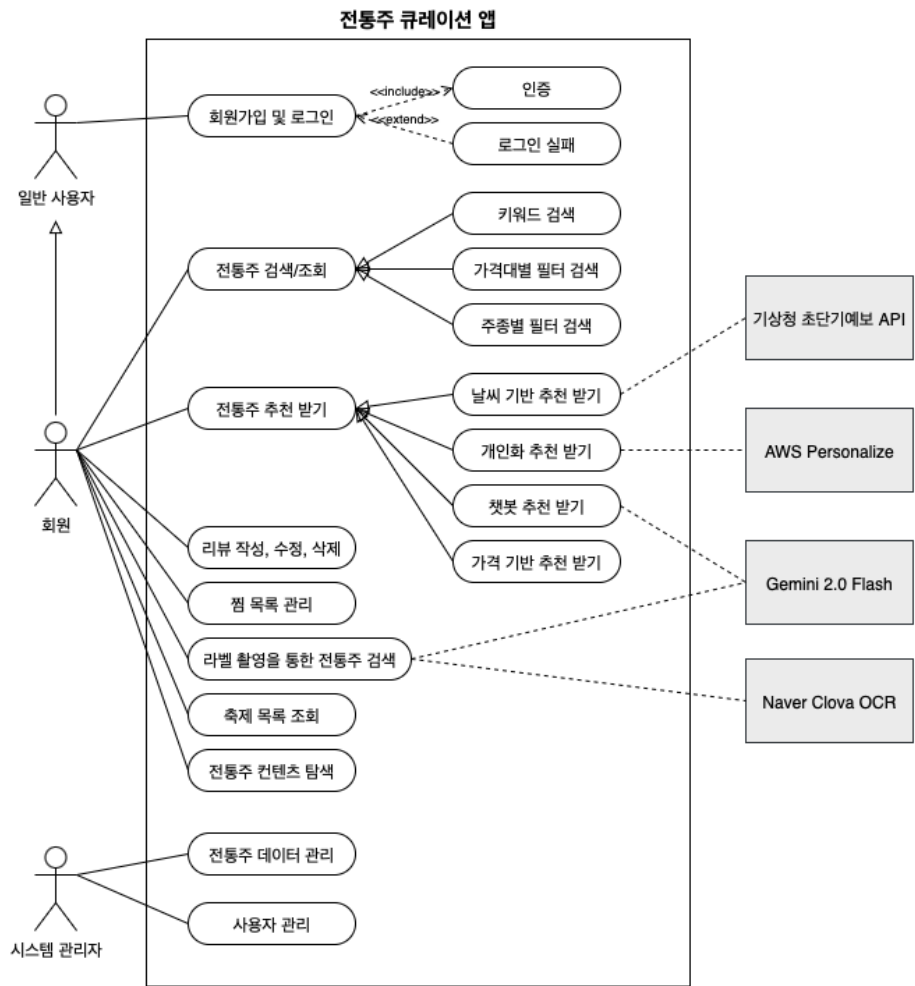
클라이언트 사용자는 19세 이상의 성인으로 전통주에 대한 관심이 있거나 구매 의사가 있는 사용자를 대상으로 한다. 한글을 원활하게 읽고 이해할 수 있으며, 스마트폰이나 웹 브라우저를 통한 기본적인 인터넷 서비스 이용에 어려움이 없는 수준의 디지털 리터러시를 갖추고 있다고 가정한다. 사용자는 터치스크린 조작, 챗봇과의 텍스트 기반 대화, 카메라를 이용한 OCR 기능 사용 등의 기본적인 모바일 앱 기능을 학습하고 활용할 수 있는 능력을 보유하고 있다고 가정한다. 또한 전통주 관련 기본 용어(도수, 지역, 원료 등)에 대한 이해가 있거나 학습 의지가 있는 사용자를 대상으로 한다. 일반적으로 19세 이상 65세 이하의 대한민국 국민을 주요 사용자로 가정하며, 특히 20-30대의 전통주에 관심있는 사용자층을 주요 타겟으로 한다.

3.1.2 제약조건

본 프로젝트는 다음의 제약조건을 준수하여 설계 및 구현한다.

- 기술 스택 제약: Express.js 기반 백엔드, React Native 기반 프론트엔드, MongoDB 데이터베이스 사용
- 외부 API 의존성: Gemini API, 공공데이터포털 기상청 초단기예보 조회 API, AWS Personalize 추천 엔진 등 외부 서비스 연동 필수
- 라이선스 제약: 오픈소스 기반 개발을 원칙으로 하되, 필수 유료 API는 예외적으로 사용
- 성능 제약: 모바일 앱 환경에서의 실시간 응답성 확보 필요
- 데이터 제약: 개인정보보호법 준수, 사용자 데이터 보안 처리 의무
- 개발 기간 제약: 캡스톤디자인프로젝트 그로쓰 과목 일정에 맞춘 단계별 개발 진행

3.1.3 기능적 요구사항



유스케이스 이름	회원가입 및 로그인
액터	일반 사용자
시작 조건	1. 모바일 앱이 설치되어 있어야 한다. 2. 네트워크 연결이 되어 있어야 한다. 3. 구글 또는 카카오 계정이 있어야 한다.
기본 흐름	1. 사용자가 앱을 실행한다. 2. 로그인 화면에서 '구글 로그인' 또는 '카카오 로그인' 버튼을 선택한다.

	3. 시스템은 선택된 소셜 로그인 프로바이더(구글/카카오) 인증 페이지로 이동한다. 4. 사용자는 해당 소셜 계정으로 로그인한다. 5. 소셜 프로바이더에서 인증이 성공하면 사용자 정보(이메일)를 반환한다. 6. 시스템은 해당 이메일로 기존 계정이 있는지 확인한다. 7. 기존 계정이 없는 경우 새 계정을 생성하고 토큰을 발급한다. 기존 계정이 있는 경우 토큰을 발급하고 로그인 완료한다. 8. 사용자는 메인 화면으로 이동한다.
대안 흐름	5a. - 인증 실패 시, 시스템은 "인증에 실패했습니다. 다시 시도해주세요" 메시지를 표시한다.
종료 조건	로그인이 성공하면, 사용자는 회원 상태가 되어 모든 앱 기능을 이용할 수 있게 된다. 액세스 토큰이 발급되어 세션이 유지된다.

유스케이스 이름	전통주 검색/조회
액터	회원
시작 조건	1. 사용자가 로그인된 상태여야 한다. 2. 네트워크 연결이 되어 있어야 한다.
기본 흐름	1. 사용자가 검색 화면에 접근한다. 2. 사용자는 검색 방법을 선택한다. 2.1. 일반 검색의 경우: 입력창에 전통주명을 입력한다. 2.2. 키워드 검색의 경우: 키워드 목록 버튼들 중 선택한다. 2.3. 필터 검색의 경우: 다음 필터링을 적용한다. - 가격대별 필터 - 주종별 필터 3. 시스템은 선택된 조건에 맞는 전통주 목록을 표시한다. 4. 사용자는 목록에서 특정 전통주를 선택한다. 5. 시스템은 선택된 전통주의 상세 정보를 표시한다.
대안 흐름	3a. - 검색 결과가 없는 경우: "검색 결과가 없습니다" 메시지를 표시한다.
종료 조건	검색이 완료되면 사용자가 원하는 전통주 정보를 확인할 수 있다. 상세 페이지에서 리뷰 확인, 찜하기 등의 추가 기능을 이용할 수 있다.

유스케이스 이름	전통주 추천 받기
액터	회원
시작 조건	1. 사용자가 로그인된 상태여야 한다. 2. 네트워크 연결이 되어 있어야 한다.
기본 흐름	1. 사용자가 앱을 실행하여 메인 화면에 접근한다. 2. 시스템은 자동으로 추천 전통주 목록을 표시한다. 2.1. 취향 기반 추천 - 최초 1회 선호도 테스트(단맛/쓴맛/신맛/바디감/탄산/도수)를 통해 사용자 취향을 DB에 저장한다. - 저장된 선호도를 기반으로 맞춤 전통주를 추천한다. 2.2. 날씨 기반 추천 - 시스템이 현재 위치 기반으로 기상청 초단기예보 API에서 날씨 정보를 가져온다. - 날씨 기반 추천 섹션에 추천 목록을 표시한다. 2.3. 개인화 추천 - 시스템이 AWS Personalize를 통해 사용자의 조회 기록, 찜 목록, 리뷰 평점을 분석한다. - 개인화 추천 섹션에 추천 목록을 표시한다. 2.4. 가격 기반 추천 - 사용자가 가격 조건(예: 3만원 이하)을 설정한다. - 가격 기반 추천 섹션에 추천 목록을 표시한다. 3. 사용자는 추천된 목록에서 관심 있는 전통주를 선택한다. 4. 챗봇 추천의 경우: 4.1. 사용자가 별도의 챗봇 화면에 접근한다. 4.2. 원하는 조건을 자유롭게 텍스트로 입력한다. 4.3. 시스템이 Gemini API를 통해 맞춤형 전통주를 추천한다.
대안 흐름	2.2a. - 위치 권한을 거부한 경우: 날씨 정보 섹션에 위치 권한 허용을 요청하는 메시지를 표시한다.

종료 조건	메인 화면에 추천 전통주 목록이 표시되고, 사용자가 취향에 맞는 전통주를 확인할 수 있다.
-------	--

유스케이스 이름	리뷰 작성, 수정, 삭제
액터	회원
시작 조건	1. 사용자가 로그인된 상태여야 한다. 2. 네트워크 연결이 되어 있어야 한다.
기본 흐름	1. 리뷰 작성의 경우 1.1. 사용자가 전통주 상세 페이지에 접근한다. 1.2. 리뷰 작성 버튼을 선택한다. 1.3. 평점을 선택한다(1점~5점). 1.4. 텍스트 리뷰를 작성한다. 1.5. 원하는 경우 사진을 첨부한다. (사진 앱에서 선택) 1.6. 리뷰 등록 버튼을 선택한다. 1.7. 시스템이 리뷰를 저장하고 해당 전통주 페이지에 표시한다. 2. 리뷰 수정/삭제의 경우 2.1. 사용자가 마이페이지에 접근한다. 2.2. “작성한 리뷰” 버튼을 선택한다. 2.3. 사용자가 작성한 리뷰 목록이 표시된다. 2.4. 수정할 리뷰의 수정 버튼을 선택하거나 삭제 버튼을 선택한다. - 수정의 경우: 평점, 텍스트, 사진을 수정 후 수정 완료 버튼을 선택한다. - 삭제의 경우: 삭제 확인 팝업에서 확인을 선택한다. 2.5. 시스템이 변경사항을 저장한다.
대안 흐름	1a. - 이미 해당 전통주에 리뷰를 작성한 경우 오류 메시지를 표시한다. 1b. - 별점을 선택하지 않은 경우 오류 메시지를 표시한다. 1c. - 제목 또는 내용을 입력하지 않은 경우 오류 메시지를 표시한다. 2a. - 작성한 리뷰 목록 조회에 실패한 경우 오류 메시지를 표시한다.
종료 조건	리뷰 작성/수정/삭제가 완료되면 해당 전통주 페이지와 마이페이지의 리뷰 목록이 업데이트된다. 다른 사용자들도 작성된 리뷰를 확인할 수 있다.

유스케이스 이름	찜 목록 관리
액터	회원
시작 조건	1. 사용자가 로그인된 상태여야 한다. 2. 네트워크 연결이 되어 있어야 한다.
기본 흐름	1. 찜 추가의 경우 1.1. 사용자가 전통주 목록 페이지 또는 상세 페이지에서 하트 버튼을 선택한다. 1.2. 시스템이 해당 전통주를 사용자의 찜 목록에 추가한다. 1.3. 하트 버튼이 채워진 상태로 변경된다. 2. 찜 해제의 경우 2.1. 사용자가 이미 찜한 전통주의 채워진 하트 버튼을 다시 선택한다. 2.2. 시스템이 해당 전통주를 사용자의 찜 목록에서 제거한다. 2.3. 하트 버튼이 빈 상태로 변경된다. 3. 찜 목록 조회의 경우 3.1. 사용자가 마이페이지에 접근한다. 3.2. “찜한 전통주” 버튼을 선택한다. 3.3. 시스템이 사용자가 찜한 전통주 목록을 표시한다. 3.4. 사용자는 목록에서 특정 전통주를 선택하여 상세 정보를 확인할 수 있다.
대안 흐름	3a. - 찜한 전통주가 없는 경우: “찜한 전통주가 없습니다” 메시지를 표시한다.
종료 조건	찜 추가/해제가 완료되면 하트 버튼 상태가 변경되고, 마이페이지의 찜 목록이 업데이트된다. 사용자는 언제든지 찜한 전통주 목록을 확인할 수 있다.

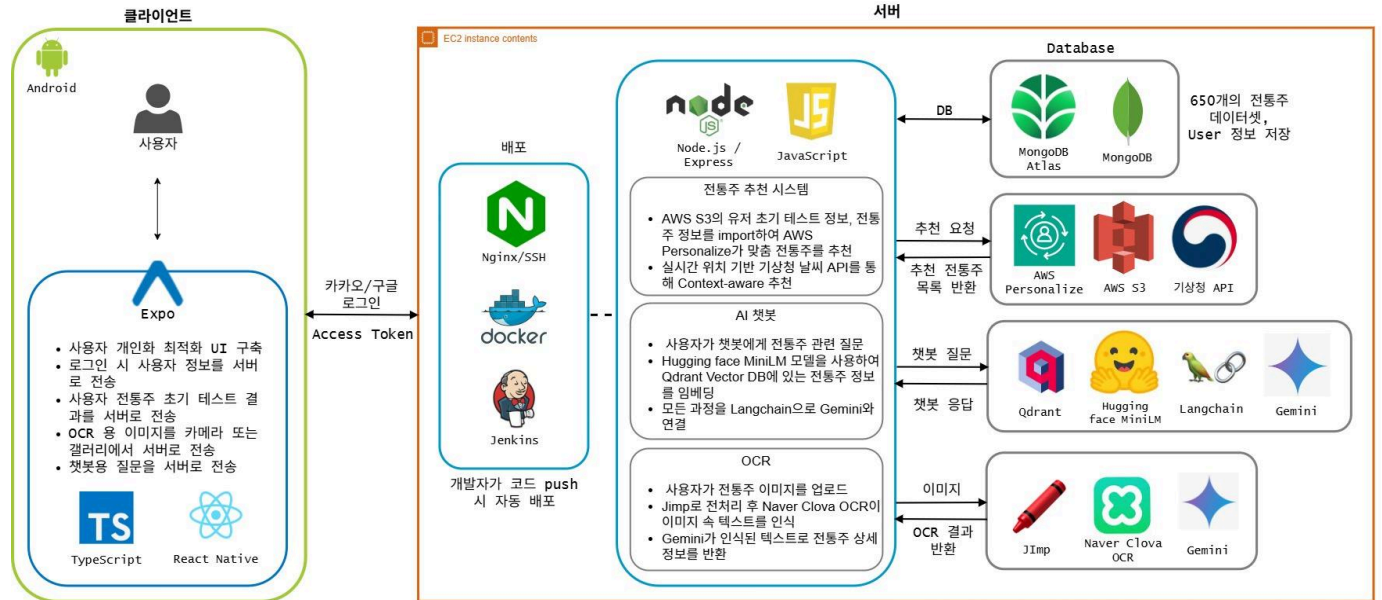
유스케이스 이름	라벨 촬영을 통한 전통주 검색
----------	------------------

액터	회원
시작 조건	1. 사용자가 로그인된 상태여야 한다. 2. 네트워크 연결이 되어 있어야 한다.
기본 흐름	1. 사용자가 OCR 검색 화면에 접근한다. 2. 사용자는 이미지 업로드 방식을 선택한다. 2.1. 사진 앱 업로드의 경우 - 사진 업로드 버튼을 선택한다. 2.2. 사진 촬영 업로드의 경우 - 카메라 화면이 활성화되고 전통주 라벨을 촬영한다. 3. 선택/촬영된 이미지가 화면에 표시된다. 4. 분석 버튼을 선택한다. 5. 시스템이 이미지를 Naver Clova OCR API로 전송하여 텍스트를 추출한다. 6. 추출된 텍스트 정보를 Google Gemini API로 전송하여 분석한다. 7. Gemini가 전통주 정보를 분석하여 전통주 정보를 제공한다. 8. 시스템이 전통주 상세 페이지를 사용자에게 표시한다.
대안 흐름	2.1a. - 사진 앱 접근 권한이 거부된 경우: 사진 앱 권한 허용을 요청하는 메시지를 표시한다. 2.2a. - 카메라 권한이 거부된 경우: 카메라 권한 허용을 요청하는 메시지를 표시한다.
종료 조건	OCR 검색이 완료되면 사용자가 촬영한 라벨에 해당하는 전통주 정보를 확인할 수 있다. 매칭된 전통주에서 상세 조회, 찜하기 등의 추가 기능을 이용할 수 있다.

유스케이스 이름	축제 목록 조회
액터	회원
시작 조건	1. 사용자가 로그인된 상태여야 한다. 2. 네트워크 연결이 되어 있어야 한다.
기본 흐름	1. c사용자가 축제 정보 화면에 접근한다. 2. 사용자가 조회하고자 하는 연도를 선택한다. 3. 시스템이 해당 연도의 축제 목록을 시작일 순으로 정렬하여 표시한다.
대안 흐름	3a. - 데이터베이스 조회 오류 발생 시 오류 메시지를 표시한다.
종료 조건	축제 목록 조회가 완료되면 사용자가 원하는 연도의 축제 목록 정보를 확인할 수 있다.

유스케이스 이름	전통주 콘텐츠 탐색
액터	회원
시작 조건	1. 사용자가 로그인된 상태여야 한다. 2. 네트워크 연결이 되어 있어야 한다.
기본 흐름	1. 사용자가 액티비티 탭에 접근한다. 2. 시스템이 전통주 관련 콘텐츠를 로드한다. 3. 다음과 같이 다양한 카테고리의 콘텐츠가 카드 형식으로 표시된다: - 전통주 입문 이야기 (흥미로운 전통주 소개) - 전통문화 속 전통주 (역사 속 재미있는 이야기) - 전통주 용어 탐구 (단어의 어원 및 유래) - 전통주 상식 (알아두면 좋은 정보) 4. 사용자는 화면을 스크롤하면서 다른 카드를 계속 탐색하거나 페이지를 종료할 수 있다.
대안 흐름	-
종료 조건	사용자가 전통주 관련 콘텐츠를 확인한 후 다른 탭으로 이동하거나 앱을 종료할 수 있다.

3.2. 전체시스템 구성도



3.2.1 전체 SW 구조

- 사용자가 클라이언트 측에서 보낸 요청을 서버에서 받아 다양한 기능에 대한 응답을 다시 클라이언트로 전송한다.
- 클라이언트에서는 React Native, TypeScript를 사용해 화면을 띄우고 서버로 정보를 전송한다.
- 서버에서는 EC2로 서버를 띄우고, Jenkins로 CI/CD를 진행한다.
- 데이터베이스는 MongoDB를 사용하고, MongoDB Atlas로 배포한다.

3.2.2 주요 기능에 대한 제어/데이터 흐름

① 전통주 추천

1. 사용자가 로그인 후, 메인 화면에 접속한다.
2. 클라이언트에서 사용자의 로그인으로 받은 accessToken을 서버로 전송한다.
3. 서버에서 받은 accessToken으로 사용자 id를 식별한다.
4. 서버에서 AWS Personalize로 사용자의 전통주 맞춤 추천 정보를 요청한다.
5. AWS Personalize에서는 필요한 정보(전통주 취향 테스트 정보, 전통주 상세 정보)를 S3에서 학습한다.(최초 1회)
6. 추가적인 사용자의 인터랙션(조회, 찜, 리뷰 등)이 있다면 AWS Personalize의 이벤트 트래커가 즉시 학습해 추천에 반영한다.
7. AWS Personalize가 추천 목록을 반환한다.
8. 오늘의 날씨에 따라(기온, 강수량) 추천 정보를 필터링해 클라이언트로 전송한다.
9. 사용자의 메인 화면에 추천 정보가 표시된다.

② RAG 기반 챗봇

1. 650여개의 전통주 상세 정보를 Qdrant 벡터 DB에 Hugging face MiniLM 임베딩 모델이 임베딩하여 저장해 놓는다.
2. 질문 입력
 - a. 사용자가 챗봇 화면에 접속해 궁금한 질문을 입력한다.
 - b. 클라이언트에서 사용자의 질문을 서버로 전송한다.
 - c. 서버에서 사용자 질문을 받아 공통된 부분을 제거하는 정제 과정을 거친다.
 - d. 서버에서 정제된 질문을 MiniLM 모델로 임베딩해 숫자 벡터로 변환한다.
 - e. 질문의 숫자 벡터와 유사한 벡터를 Qdrant 벡터 DB에서 검색한다.
 - f. 검색된 유사 벡터의 데이터를 프롬프트와 함께 Gemini로 넘겨준다.
 - g. Gemini가 받은 데이터를 기반으로 응답을 json 형식으로 반환한다.
 - h. 클라이언트에서 받은 json 응답을 채팅 형태로 사용자의 화면에 띄운다.
3. 대화 이력 조회
 - a. 클라이언트에서 넘겨준 accessToken을 기반으로 서버에서 사용자의 id를 해석해 대화 내용을 MongoDB에 저장한다.
 - b. 사용자가 화면 진입 시, 서버에서 id를 기반으로 저장된 대화 이력을 클라이언트로 전송한다.
 - c. 클라이언트에서 받은 대화 내용을 채팅 형식으로 사용자 화면에 띄운다.
4. 이러한 전체 과정을 Langchain으로 조율하고 관리한다.

③ OCR

1. 사용자가 OCR 버튼을 클릭해 OCR 화면에 접속한다.
2. 사용자가 정보를 원하는 사진을 직접 촬영하거나, 갤러리에서 선택한다.
3. 클라이언트는 선택된 이미지를 **multi-form** 형식으로 서버에 전송한다.
4. 서버에서는 이미지를 **Jim** 모듈로 전처리하고, **Naver Clova OCR**로 텍스트를 추출한다.
5. 추출된 텍스트를 **Gemini**로 전송해 전통주 이름을 분석해 상세 정보를 반환한다.
6. 반환된 상세 정보를 클라이언트가 화면에 띄워준다.

3.3. 주요 엔진 및 기능 설계

① AWS Personalize 모듈 구현 과정

1. AWS에서 Personalize를 사용할 주체인, IAM 유저를 생성하기 -> 서버에서 AWS Personalize를 사용 가능하다.
 - a. 다음 권한들을 부여한다.
 - i. AmazonPersonalizeFullAccess
 - ii. AmazonS3FullAccess
 - iii. AmazonEC2FullAccess
 - iv. AmazonRDSFullAccess
 - v. CloudWatchFullAccess
 - b. access key 생성하기
2. AWS S3 버킷 생성하기 -> 개인화 추천 모델의 학습에 필요한 데이터들
 - a. S3 정책 추가하기

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Sid": "AllowPersonalizeServiceAccess",  
6       "Effect": "Allow",  
7       "Principal": {  
8         "Service": "personalize.amazonaws.com"  
9       },  
10      "Action": [  
11        "s3:GetObject",  
12        "s3:ListBucket"  
13      ],  
14      "Resource": [  
15        "arn:aws:s3:::sool-s3-bucket",  
16        "arn:aws:s3:::sool-s3-bucket/*"  
17      ]  
18    }  
19  ]  
20 }  
21
```

- b. csv 데이터셋 업로드하기
 - i. users.csv 업로드(유저 id, 전통주 취향 테스트 정보)
 - ii. items.csv 업로드(전통주 650여개 상세 정보)
 - iii. interactions.csv 업로드(유저의 전통주 관련 행동 기록 정보)
3. AWS Personalize 리소스 생성하기 -> 추천용 모델 생성
 - a. Dataset 그룹 생성: E-commerce Domain 선택한다.
 - b. S3에 저장된 csv 파일을 import한다.
 - c. Solution을 생성해 추천기를 생성 한다.
 - i. 사용자 맞춤 추천을 위해 User custom resources를 선택한다.
 - ii. 상품 추천을 위해 Item recommendation를 선택한다.
 - iii. 레시피로 최신 모델인 aws-user-personalization-v2를 선택한다.
 - iv. 이벤트별 가중치를 선택한다.(클릭: 0.5, 찜: 1.0, 리뷰: 0.5*별점)
 4. Campaign 생성하기: 서버에서 AWS Personalize api 사용이 가능하다.
 5. 서버 코드
 - a. 개인화 추천 recommend.service.js
 - i. getRecommendations() 함수에서 userId, numResults(반환할 전통주 개수)를 파라미터로 받는다.
 - ii. GetRecommendationsCommand()에서 campaignArn을 사용해 추천 api와 연결한다.
 - iii. personalizeClient.send()에서 추천 전통주의 id 리스트를 받아온다.
 - iv. MongoDB에서 Alcohol.find(id)로 전통주 id를 사용해 전통주 상세 정보(이름, 도수, 이미지)를 받아온다.
 - v. 전통주 상세 정보 목록을 원하는 객체 형태로 return한다.
 - b. 사용자 인터랙션 반영 personalize.service.js
 - i. PersonalizeEventsClient()를 awsAccessKey를 사용해 이벤트 발생기를 생성한다.
 - ii. sendBookmarkEvent는 사용자 인터랙션 중 '찜' 발생 시 PutEventsCommand()에서 userId, 전통주 id, eventType을 받아 AWS Personalize에 이벤트를 전송한다.
 - iii. sendViewDetailEvent는 사용자 인터랙션 중 '조회' 발생 시 PutEventsCommand()에서 userId, 전통주 id, eventType을 받아 AWS Personalize에 이벤트를 전송한다.
 - iv. sendReviewEvent는 사용자 인터랙션 중 '리뷰' 발생 시 PutEventsCommand()에서 userId, 전통주 id, 리뷰 별점(rating), eventType을 받아 AWS Personalize에 이벤트를 전송한다.

② RAG 기반 챗봇 모듈 구현 과정

- Qdrant Vector DB
 - 650여개의 전통주 상세 정보(index, 이름, 도수, 단맛, 신맛, 바디감, 탄산, 어울리는 음식, 키워드 등)를 저장한다.
 - 전통주 상세 정보를 임베딩하여 숫자 벡터로 변환해 저장한다.
- Hugging face MiniLM 임베딩 모델
 - Qdrant Vector DB에 저장할 전통주 정보를 숫자 벡터로 임베딩한다.
 - 사용자의 질문을 숫자 벡터로 임베딩한다.
- Gemini LLM 모델
 - 질문과 유사한 전통주 목록을 **context** 변수로 받는다.
 - 사용자의 정제된 질문을 **question** 변수로 받는다.
 - 프롬프트의 역할 부여/오류 제어/반환 형식에 따라 **json** 형태로 응답한다.
- Langchain 모델
 - 챗봇의 질문-응답 전체 흐름을 관장한다.
 - langchain이 동작시키는 전체 흐름
 - 질문이 들어오면 공통된 단어를 제외하도록 정제한다.
 - 정제된 질문 텍스트를 임베딩 모델을 사용해 벡터로 변환한다.
 - 질문 벡터와 가장 유사한 Qdrant의 전통주 벡터를 검색한다.
 - 검색된 유사한 전통주 정보를 LLM 모델로 넘긴다.
 - LLM 모델이 반환한 **json** 응답을 클라이언트로 반환한다.
- 서버 코드
 - Qdrant 초기 임베딩 코드
 - vectorstore에서 **init.js**가 **embedding.service.js** 코드를 실행한다.
 - 임베딩 코드에서 **XenovaEmbeddings** 클래스를 정의한다.
 - **getEmbedder()**에서 "Xenova/all-MiniLM-L6-v2" 임베딩 모델을 호출한다.
 - **embedQuery(text)**에서 쿼리 임베딩을 정의한다.
 - **embedDocuments(texts)**에서 문서 임베딩을 정의한다.
 - **cleanPayload(obj)**: 문자열 정제 함수를 정의한다.
 - **initEmbedding()**으로 csv의 데이터를 벡터로 임베딩해 Qdrant Vector DB에 저장한다.
 - **QdrantClient()** 생성자로 Qdrant DB 객체를 생성한다.
 - **XenovaEmbeddings()** 생성자로 임베딩 객체를 생성한다.
 - **fetch DELETE method**로 기존 Qdrant 컬렉션을 삭제한다.
 - **fetch PUT method**, 384 차원으로 새 Qdrant 컬렉션을 생성한다.
 - **cleanPayload()**로 문자열을 정제하고, **embeddings.embedQuery()**로 텍스트 임베딩한다.
 - **fetch PUT method**로 데이터를 Qdrant에 1개씩 업로드한다.
 - Langchain 챗봇 코드 **langchain.service.js**
 - 임베딩 코드에서 **XenovaEmbeddings** 클래스 정의, 생성한다.
 - **getEmbedder()**에서 "Xenova/all-MiniLM-L6-v2" 임베딩 모델을 호출한다.
 - **embedQuery(text)**에서 쿼리 임베딩을 정의한다.
 - **embedDocuments(texts)**에서 문서 임베딩을 정의한다.
 - **cleanQueryForEmbedding()**에서 질문 문자열 정제 함수를 정의한다.('술', '전통주' 등 공통 문자열 제거)
 - **cleanJsonResponse()**에서 챗봇 응답을 **json**으로 파싱하는 함수를 정의한다.
 - **ChatGoogleGenerativeAI()**에서 **geminiApiKey**를 사용해 Gemini 모델을 정의한다.
 - **ChatPromptTemplate.fromTemplate()**에서 **prompt.txt**를 path로 받아 프롬프트를 정의한다.
 - **retrieveAndFormatContext()**에서 랭체인 질문 임베딩 함수를 정의한다.
 - **cleanQueryForEmbedding()**로 질문을 정제한다.
 - **embeddings.embedQuery()**로 네이티브 임베딩을 한다.
 - **qdrant.search()**의 네이티브 Qdrant 검색으로 유사 전통주를 검색한다.
 - 검색된 전통주를 'd' 속성으로 LLM에게 전달할 형식으로 반환한다.(이름, 어울리는 음식, 키워드 등)
 - **RunnableSequence.from()**에서 랭체인 체인(LCEL) 정의하기
 - **RunnableLambda()** 생성자로 **context**와 **userQuestion** 변수를 생성한다.
 - 생성한 변수를 프롬프트에 주입한다.
 - 생성된 프롬프트를 Gemini에게 전달한다.
 - **cleanJsonResponse()**로 **json** 파싱한다.
 - **recommendSool()**에서 **userId**, 사용자 질문을 받아 랭체인 호출/실행하기
 - **recommendSoolChain.invoke()**로 정의한 LCEL 랭체인 실행한다.
 - 검색 결과가 NULL일 시 빈 배열로 반환한다.
 - **ChatLog.create()**에서 **userId**, **question**, **answer**로 대화 기록을 저장한다.
 - 제약: 임베딩 모델이 고성능 모델이 아니기 때문에 사용자의 질문을 자세하게 벡터로 구분하지 못 한다. 하지만 고성능 모델로 변경 시 EC2의 RAM 사용량이 초과되기 때문에 고성능 모델을 사용할 수 없다. 2차 방안으로 사용자의 질문 중 공통되는 부분(전통주, 추천 등)을 제외하는 식으로 질문 정제 과정을 추가하였다.

3.4. 주요 기능의 구현

3.4.1. AWS Personalize 모듈

본 절에서는 2.5 기능 리스트의 F-011(개인화 추천) 기능이 3.3에서 설명한 AWS Personalize 모듈과 어떻게 연동되어 작동하는지 설명한다. 개인화 추천 기능은 사용자가 앱 메인 화면에 접속할 때 자동으로 실행되며, 사용자의 취향과 행동 패턴에 최적화된 전통주 목록을 추천한다.

3.4.1.1. 모듈 연관성

개인화 추천 기능은 다음의 모듈들이 유기적으로 연결되어 작동한다.

- ① AWS Personalize Campaign: 3.3에서 구축한 추천 엔진의 배포 인스턴스로, 사용자 ID를 입력받아 추천 전통주 ID 목록을 반환하는 역할
- ② recommend.service.js: AWS Personalize Campaign에 추천을 요청하고, 반환된 ID 목록을 MongoDB 데이터와 결합하여 상세 정보를 생성하는 서비스 레이어
- ③ alcohol.model.js: MongoDB의 Alcohol 컬렉션과 연동하여 전통주의 상세 정보(이름, 도수, 이미지 등)를 조회하는 데이터 모델.
- ④ recommend.controller.js: 클라이언트의 HTTP 요청을 받아 서비스 레이어를 호출하고 응답을 반환하는 컨트롤러
- ⑤ personalize.service.js: 사용자의 행동(조회, 찜, 리뷰)을 AWS Personalize Events API로 전송하여 실시간으로 추천 모델을 업데이트하는 이벤트 전송 모듈

3.4.1.2. 기능 실행 흐름

3.4.1.2.1. 메인 화면 추천 표시

사용자가 로그인 후 메인 화면에 진입하면 개인화 추천 기능이 다음과 같이 실행된다.

- ① 클라이언트가 GET /recommend/:userId 엔드포인트로 요청을 전송한다. 이때 URL 파라미터로 사용자 ID가 전달된다.
- ② recommend.controller.js의 recommendController 함수가 요청을 수신하고, req.params에서 userId를 추출한다.
- ③ recommendController는 recommend.service.js의 getRecommendations(userId)를 호출한다. getRecommendations() 함수는 기본적으로 6개의 추천을 반환하도록 설정되어 있다.
- ④ getRecommendations() 함수 내부에서 GetRecommendationsCommand 객체를 생성한다. 이 객체는 3.3에서 생성한 Campaign ARN, 사용자 ID, 추천 개수를 포함한다.
- ⑤ personalizeClient.send(command)로 AWS Personalize에 추천을 요청한다. AWS Personalize는 3.3에서 S3에 업로드한 학습 데이터(users.csv, items.csv, interactions.csv)와 실시간으로 전송된 이벤트를 기반으로 해당 사용자에게 최적화된 전통주 ID 목록을 반환한다.
- ⑥ 반환된 itemList에서 itemId만 추출하여 배열로 만든다.
- ⑦ itemId가 숫자 형태("123")인지 MongoDB ObjectId 형태인지 정규식으로 판별한다. 숫자 형태는 indexIds 배열에, 그 외는 objectIdIds 배열에 분류한다.
- ⑧ Alcohol.find({ \$or: [{ index: { \$in: indexIds } }, { _id: { \$in: objectIdIds } }] })로 MongoDB에서 해당 전통주들의 상세 정보를 한번에 조회한다.
- ⑨ AWS Personalize가 제시한 추천 순서를 유지하기 위해, 원본 itemId 배열의 순서대로 MongoDB 조회 결과를 재정렬한다. map() 함수로 각 itemId에 해당하는 상세 정보를 찾아 새로운 배열을 생성한다.
- ⑩ 클라이언트가 사용하기 편한 형태로 데이터를 변환한다. DB의 alcoholName 필드는 name으로, imageUrl은 image로 변경하여 { index, name, degree, image } 형태의 객체 배열을 만든다.
- ⑪ recommendController가 res.status(200).json(orderedAndFormattedResults)로 최종 추천 목록을 클라이언트에 반환한다.
- ⑫ React Native 클라이언트는 받은 배열을 메인 화면의 맞춤 추천 섹션에 표시한다.

3.4.1.2.2. 사용자 행동 추적 및 추천 업데이트

사용자가 추천된 전통주와 상호작용하면 해당 행동이 실시간으로 AWS Personalize에 반영된다.

① 전통주 상세 페이지 조회 시 (F-017 기능)

사용자가 추천 목록에서 특정 전통주 카드를 터치하여 상세 페이지로 이동하면, alcohol.controller.js의 getAlcoholDetail 함수가 실행된다. 이 함수는 상세 정보를 조회한 후, personalize.service.js의 sendViewDetailEvent(userId, itemId)를 호출한다. sendViewDetailEvent() 함수는 PutEventsCommand를 생성하여 다음 정보를 포함한다.

- trackingId: 환경변수에 저장된 AWS Personalize Tracking ID
- userId: 현재 사용자의 ID
- sessionId: "view-\${userId}" 형태로 조회 세션임을 표시
- eventList: [{ eventType: "view_detail", sentAt: new Date(), itemId }]

이 커맨드가 personalizeEventsClient.send()로 전송되면, AWS Personalize는 해당 이벤트를 모델에 반영한다. 3.3에서 설정한 가중치(조회 0.5)에 따라, 사용자가 조회한 전통주와 유사한 속성을 가진 다른 전통주들의 추천 확률이 상승한다.

② 전통주 찜하기 시 (F-019 기능)

사용자가 상세 페이지에서 하트 버튼을 눌러 찜하기를 실행하면, bookmark.controller.js의 addBookmark 함수가 실행된다. 이 함수는 MongoDB의 Bookmark 컬렉션에 찜 기록을 저장한 후, personalize.service.js의 sendBookmarkEvent(userId, itemId)를 호출한다. sendBookmarkEvent() 함수는 PutEventsCommand를 생성하여 다음 정보를 포함한다.

- trackingId: AWS Personalize Tracking ID
- userId: 현재 사용자의 ID
- sessionId: "bookmark-\${userId}" 형태로 찜 세션임을 표시
- eventList: [{ eventType: "bookmark", sentAt: new Date(), itemId }]

찜 이벤트는 3.3에서 설정한 가중치 1.0으로 전송되어, 조회 이벤트(0.5)보다 2배 강한 선호도 신호를 AWS Personalize에 전달한다. 이는 사용자가 해당 전통주에 대해 명확한 관심을 표현했음을 의미한다.

③ 리뷰 작성 시 (F-023 기능)

사용자가 상세 페이지에서 별점과 텍스트 리뷰를 작성하면, `review.controller.js`의 `createReview` 함수가 실행된다. 이 함수는 MongoDB의 `Review` 컬렉션에 리뷰를 저장한 후, `personalize.service.js`의 `sendReviewEvent(userId, itemId, rating)`를 호출한다. 여기서 `rating`은 사용자가 부여한 별점(1~5)이다.

`sendReviewEvent()` 함수는 `PutEventsCommand`를 생성하여 다음 정보를 포함한다.

- `trackingId`: AWS Personalize Tracking ID
- `userId`: 현재 사용자의 ID
- `sessionId`: "review-`{userId}`" 형태로 리뷰 세션임을 표시
- `eventList`: [{ `eventType`: "review", `sentAt`: new Date(), `itemId`, `properties`: JSON.stringify({ `rating` }) }]

리뷰 이벤트는 3.3에서 설정한 가중치 0.5로 전송된다.

3.4.2. RAG 기반 챗봇 모듈

본 절에서는 2.5 기능 리스트의 F-032(전통주 Q&A) 기능이 3.3에서 설명한 RAG 기반 챗봇 모듈과 어떻게 연동되어 작동하는지 설명한다. 전통주 Q&A 기능은 사용자가 챗봇 화면에 진입하여 질문을 입력하면, RAG 기반 챗봇 모듈이 사용자의 입력에서 날씨, 감정, 음식 등의 맥락 정보를 분석하여 이와 어울리는 전통주를 RAG에서 뽑아 추천한다.

3.4.2.1. 모듈 연관성

전통주 Q&A 기능은 다음의 모듈들이 유기적으로 연결되어 작동한다.

- ① **Langchain** 모델: 챗봇 전체의 질문-응답 흐름을 관장하는 모델로, 사용자의 입력 텍스트를 정제하고, 이를 기반으로 검색된 Qdrant 전통주 벡터를 LLM 모델로 전달하여 이에 대응하는 json 응답을 클라이언트로 반환하는 역할
- ② **Hugging face MiniLM** 임베딩 모델: Qdrant Vector DB에 저장하기 위해 650여개의 전통주 상세 정보(index, 이름, 도수, 단맛, 신맛, 바디감, 탄산, 어울리는 음식, 키워드 등)를 숫자 벡터로 임베딩하고, 사용자의 질문 또한 숫자 벡터로 임베딩 하는 역할
- ③ **Qdrant Vector DB**: Hugging face MiniLM 임베딩 모델을 통해 변환된 전통주 정보를 저장해둔 데이터 베이스
- ④ **Gemini LLM** 모델: Langchain 모델로부터 전달받은 context 변수와 사용자의 정제된 질문을 question 변수로 받아 제안된 프롬프트 형식에 맞춰 json 형태로 응답을 반환하는 AI 모델

3.4.2.2. 기능 실행 흐름

3.4.2.2.1. 초기 Qdrant Vector DB 임베딩 및 구축

F-032(전통주 Q&A) 기능의 동작을 위해서 필요한 서버 단의 사전 작업으로, CSV에 있는 전통주 데이터를 Qdrant Vector DB에 임베딩 하는 과정이다.

초기 임베딩 스크립트에서는 `vectorstore`에서 `init.js`를 실행하고, 내부 `embedding.service.js`의 `initEmbedding()` 함수를 호출한다. 해당 함수가 실행되면 csv의 데이터를 벡터로 임베딩하여 Qdrant Vector DB에 저장하는 초기화 작업을 다음과 같이 순서대로 실행하게 된다.

① Qdrant 클라이언트 생성

: `embedding.service.js` 내부에서 `QdrantClient()` 함수를 실행하여 Qdrant 클라이언트를 생성한다.

② 임베딩 객체 생성

: `XenovaEmbeddings()` 생성자로 임베딩 클래스 `XenovaEmbeddings`를 인스턴스화 한다. 이때 `getEmbedder()` 함수 내부에서 "Xenova/all-MiniLM-L6-v2" 임베딩 모델을 호출한다.

③ 기존 Qdrant 컬렉션 삭제

: CSV가 변경되었거나, 모델이 변경된 경우 이전 벡터와 충돌을 막기 위해 `fetch DELETE method`로 기존 Qdrant 컬렉션을 삭제한다.

④ 데이터 정제 및 임베딩

: 전통주 csv의 각 행을 순회하며 `cleanPayload()`로 정제하고, `embedDocuments(texts)`와 `fetch PUT method`를 통해 1개씩 데이터를 임베딩한다. 그 결과, 384 차원의 새 Qdrant 컬렉션이 생성된다.

3.4.2.2.2. 실시간 전통주 Q&A(F-032), 대화 기록 저장 및 조회(F-033, F-034)

사용자가 챗봇 화면에 진입하면, 클라이언트 단에서는 다음 기본 문구를 렌더링하여 사용자의 질문을 유도한다.

기본 문구: "안녕하세요! 주전부리의 챗봇 술동이예요. 어떤 전통주를 마셔야할지 잘 모르겠다면, 저에게 질문해주세요!"

① 화면 하단의 입력창에 자연어로 질문을 입력하면, 클라이언트 단에서는 `POST method`로 사용자가 입력한 질문을 JSON 형태로 서버 API로 전송한다.

② `langchain` 챗봇 코드 `langchain.service.js`는 사용자의 입력을 받으면 `recommendSool()`를 실행하고, 내부에서 `retrieveAndFormatContext()` 함수를 호출하여 다음 RAG 기반 검색 과정을 순차적으로 실행하게 된다.

- 1) 사용자의 질문 쿼리에서 검색 성능 향상을 위해, '술', '전통주' 등의 공통 문자열 제거하기 위해 정의된 `cleanQueryForEmbedding()` 함수로 맥락 정보가 담긴 단어가 남도록 문자열을 정제한다.
- 2) `embeddings.embedQuery()` 함수를 통해 정제된 질문 문자열을 네이티브 임베딩한다.
- 3) `qdrant.search()` 함수를 통해 Qdrant 내부에 저장된 전통주 벡터들간의 유사도를 계산해 사용자의 질문에 가장 적합한 전통주를 검색한다.
- 4) 검색된 전통주를 `d` 속성으로서 이름, 어울리는 음식, 키워드 등을 담아 포매팅한다.

③ `RunnableSequence.from()`에서 정의된 `Langchain` 체인 (LCEL) 파이프라인을 따라 다음 과정에 따라 LLM 입력-결과가 파싱된다.

- 1) `RunnableLambda()` 생성자로 `context`와 `userQuestion` 변수를 생성하고, `ChatPromptTemplate` 함수에 주입하여 LLM에게

2) ChatGoogleGenerativeAI() 함수를 통해 Gemini 모델에 프롬프트를 전달하여 결과를 전달받는다.
3) 해당 결과를 cleanJsonResponse()로 json 파싱하여 서버로 다시 전달한다. 만약 Qdrant 검색 결과가 비어 있거나, LLM 응답이

④ 정상적으로 결과가 생성되었다면, `ChatLog.create()`를 호출하여 `userId`, `question`, `answer`로 포맷팅된 대화 로그를 저장한다. 이후, 사용자가 챗봇 화면에 재진입하면 해당 로그를 서버에서 불러와 렌더링한다.

(4) 소프트웨어 검증

① Google OAuth 2.0 로그인

① Google OAuth 2.0 로그인

-

- 검증 방법: 모바일 앱 환경에서 카카오 계정 로그인을 시도하고, 카카오 서버로부터 코드 수신 여부를 확인하였다.
- 검증 결과: 인가 코드 발급 과정이 정상적으로 동작하였다.
- 그림 3 Kakao 인가 코드 발급 화면

[illegible]

- 검증 방법: 로그인 성공 후 Access Token과 Refresh Token이 발급되는지 확인하고, 토큰을 디코딩하여 토큰 형식(Header.Payload.Signature) 및 payload 내용을 점검함. 또한, Postman으로 Refresh Token을 이용한 토큰 재발급을 확인하였다.
- 검증 결과: 정상적으로 Access Token 및 Refresh Token이 발급되었다.
- 그림 4 JWT 토큰 페이로드 디코딩 화면, 그림 5 Refresh Token 재발급 화면(Postman)

DECODED PAYLOAD
 http://localhost:3000/oauth/login/google

[JSON](#) CLAIMS TABLE

POST http://localhost:3000/oauth/reissue

Params
Authorization
Headers (9)
Body
Scripts ●
Settings
Cookies

☒	Accept-Encoding		gzip, deflate, br	
☒	Connection		keep-alive	
☒	Authorization		Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp...	
	Key		Value	Description

Body Cookies Headers (8) Test Results (1/1)
200 OK
• 40 ms • 874 B •
...

{ }
JSON ▼
▶ Preview ▢ Visualize ▼

```

1 {
2   "grant_type": "Bearer",
3   "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJj1c2VySWQwIjE2OGRhMQ05TEYMTRIOTdJWUzZGUzZWUiLCJwcmlCJm92aWR1ciI6Imdvb2dsZSI6InR5cGU0IjhyZWluc3MlICJpYXQiOjE3NTkxMTkzMjAsImV4cCI6MTc1OTEyMjkyYXVkiOjoic29ybC1lc2VycyI6InVnb2wtYXBkbnIuOiBzaig8-uVR0a4xOZSDAEFkz6U17oYq6PfW0v4tBF4A".
4   "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJj1c2VySWQwIjE2OGRhMQ05TEYMTRIOTdJWUzZGUzZWUiLCJwcmlCJm92aWR1ciI6Imdvb2dsZSI6InR5cGU0IjhyZWluc3MlICJpYXQiOjE3NTkxMTkxMjAsImV4cCI6MTc1OTEyMjkyYXVkiOjoic29ybC1lc2VycyI6InVnb2wtYXBkbnIuOiBzaig8-uVR0a4xOZSDAEFkz6U17oYq6PfW0v4tBF4A".
5 }
```


④ 전통주 데이터 크롤링, 정제 및 저장

- 검증 방법: 네이버 지식백과에서 전통주 관련 데이터를 크롤링하고, 수집된 데이터의 수량 및 항목을 검토함. 크롤링된 데이터를 CSV 형식으로 변환하고, 구조화된 형태(컬럼 단위)로 저장 여부를 확인하였다.
- 검증 결과: 약 600여 종의 전통주 데이터가 확보되었으며, 주요 속성(이름, 주종, 설명 등)이 수집되었다. 데이터 정제가 완료되어 CSV 파일로 안정적으로 확보되었다.
- 그림 6 csv 형태의 전통주 데이터 화면

A	B	C	D	E	F	G	H	I	J	K	L	M	N
index	alcoholName	normalizedName	foodPairing	sweetness	sourness	freshness	body	degree	alcoholType	keyword	volume	price	manufacturer
0	1000의 막걸리 프	1000의막걸리프	여름에는 시원하게	3	4	3	3	5%	탁주		750ml	₩3,500 (가격은 ₩ (주)국순당	
1	1000의 유산균막걸	1000의유산균막걸	새콤함과 달콤함의	3	5	5	3	5%	탁주		750ml	₩3,200 (가격은 ₩ (주)국순당	
2	1932 새싹망종 햅	1932새싹망종햅	케이크 등 디저트 &	2	2	3	5	6%	생막걸리		900ml	500ml-W15,000((주)1932 포천일동	
3	1932 새싹망종 햅	1932새싹망종햅	데저 갈비 등 구운	3	1	1	4	6%	생막걸리		900ml	500ml-W15,000((주)1932 포천일동	
4	1950 씨유포터담	1950씨유포터담	김치 와인의 상큼함	2	2	1	3	12%	과실주		750ml	₩42,000 (가격은 ₩ 농업회사법인(주)K	
5	264 청포도 와인 쥬	264청포도와인쥬	광어 세비체, 찐삼	1	4	4	2	12.50%	과실주	과일류	375ml / 750ml	₩22,000 / ₩33, (264청포도 와인	
6	33JU	33ju	산양산삼 고유의 풍	0	0	0	0	18%	일반증류주		500ml / 200ml	₩49,000 / ₩23, (농업회사법인 33	
7	52C	52c	급탕구이와 잘 어우	0	0	0	0	17.50%	일반증류주		500ml	₩10,000 (가격은 ₩ 농업회사법인(주)K	
8	DMZ 240km	dmz240km	회무침과 잘 어울린	0	0	0	0	25%	기타주류		450ml	가격은 판매처 별로 달라짐	
9	G12 굴디락스	g12굴디락스	닭백숙, 오리백숙, &	2	2	2	3	12%	탁주		500ml	₩12,000 (가격은 ₩ 가평 우리술	
			- 애플: 가벼운 사 - 진저: 상큼하고 - 만다린: 첫 만남 - 오스카트: 청포도 - 피치: 감미로운 &										
10	The 첫물수룩 시라	the첫물수룩시라	- 아메리카노: 아메	4	2	3	2	피치 14%, 오스카: 과실주			360ml	₩12,000(360ml)* (주)한국애플리즈	
11	가덕 순발 막걸리	가덕순발막걸리	미나리전, 고기 반드	3	3	3	3	6%	탁주		750ml	₩13,500(750ml)* 농업회사법인 명세	
12	가을빛 레드와인	가을빛레드와인	다만 반년감은 적은	2	3	3	2	14%	과실주		750ml	₩35,000 (가격은 ₩ 예인와인	
13	가평 첫 막걸리	가평첫막걸리	반찬으로 나오는 유	2	2	3	3	6%	탁주		750ml	₩1,450 (가격은 ₩ 가평 우리술	

⑤ Gemini API 챗봇 연동

- 검증 방법: Postman을 이용하여 Gemini 2.0 Flash 모델에 질의 요청을 보내고, 응답 데이터가 정상적으로 수신되는지 확인하였다.
- 검증 결과: 질의에 대한 적절한 챗봇 응답이 정상적으로 반환되었다.
- 그림 7 Gemini API 응답 화면

```

{
  "result": [
    {
      "name": "해장합살생막걸리",
      "summary": "청량감이 특징인 합살 막걸리",
      "reason": "김치전의 느끼함을 잡아주고, 청량한 맛이 조화롭게 어울립니다.",
      "image": "이미지 URL은 제공되지 않았습니다.",
      "detailPage": "상세페이지 URL은 제공되지 않았습니다."
    },
    {
      "name": "찰막걸리",
      "summary": "청량감이 있는 막걸리",
      "reason": "김치전의 매콤함과 느끼함을 찰막걸리의 청량함이 깔끔하게 잡아줍니다.",
      "image": "이미지 URL은 제공되지 않았습니다.",
      "detailPage": "상세페이지 URL은 제공되지 않았습니다."
    }
  ]
}

```

⑥ AWS Personalize 추천 시스템

- 검증 방법: 크롤링된 데이터를 기반으로 AWS Personalize에 모델을 학습시키고, 추천 결과가 생성되는지 확인하였다.
- 검증 결과: 개인화 추천 결과가 정상적으로 도출되었다.
- 그림 8 AWS Personalize 추천 결과 화면

```

PS C:\Users\yookr\HabitAI\Server\recommend> node .\aws_personalize.js
Recommended items for user 1:
[
  { itemId: '15', score: 0.1008727 },
  { itemId: '3', score: 0.0779402 },
  { itemId: '9', score: 0.0645529 },
  { itemId: '2', score: 0.0561479 },
  { itemId: '17', score: 0.0537974 },
  { itemId: '6', score: 0.0451749 }
]

```

⑦ OCR 기능 (Naver Clova OCR)

- 검증 방법: Postman을 이용해 이미지 파일을 업로드하여 OCR API로 텍스트를 추출하고, 인식 정확성을 확인하였다.
- 검증 결과: 전통주 인식율은 높지만 전통주명 필드가 잘리는 오류가 존재한다.
- 그림 9 OCR 결과 화면

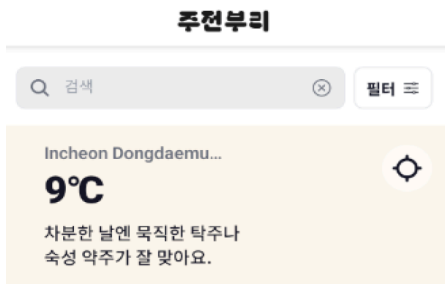
```

{
  "이름": "주오메",
  "도수": "11%",
  "단맛": "중간",
  "신맛": "약",
  "쓴맛": "약",
  "어울리는 음식": "전, 육류",
  "구성 원재료": "미상 (병 표기 정보 부족)"
}

```

⑧ 기상청 초단기예보 API (사용자 현재 위치 기반 날씨정보)

- 검증 방법: **React Native** 앱의 메인 화면에서 기상청 초단기예보 **API**를 호출하여 현재 위치 기반의 날씨 데이터를 요청하고, 반환된 응답(기온, 강수형태 등)이 **UI**에 정상적으로 렌더링되는지 안드로이드 에뮬레이터 화면을 통해 확인하였다.
- 검증 결과: 초단기예보 정보가 반환되어 안드로이드 에뮬레이터에 표시되었다.
- 그림 10 날씨 정보가 표시되는 안드로이드 에뮬레이터 화면



(5) 소프트웨어 평가

5.1. 평가항목 선정

SW	평가 항목	설명	선정 사유
챗봇	검색 정확도	사용자의 질의 의도에 부합하는 전통주 정보(문서)가 벡터 DB(Qdrant)에서 올바르게 추출되었는지를 평가한다.	프로젝트의 목표인 '전통주 입문자를 위한 큐레이션'을 달성하기 위해서는 엉뚱한 술이 아닌, 사용자가 원하는 특징(예: 달달한, 도수 높은)을 가진 술을 정확히 찾아내는 것이 서비스의 근간이 되기 때문이다.
	답변 충실도	생성 모델(Gemini)이 검색된 문맥(Context) 내의 사실에 기반하여 답변했는지, 아니면 없는 내용을 지어냈는지(환각 현상)를 평가한다.	정보 접근성이 낮은 입문자들에게 잘못된 정보(예: 없는 맛이나 효능)를 제공할 경우 서비스 신뢰도가 치명적으로 하락하므로, 생성형 AI의 고질적인 문제인 환각 현상을 최소화했는지 검증이 필요하다.
	응답 속도	사용자 질문 입력 시점부터 답변 출력이 완료될 때까지 걸리는 시간을 측정한다.	RAG 프로세스(임베딩 -> 검색 -> 생성)는 일반적인 DB 조회보다 시간이 오래 걸린다. 실시간 챗봇 서비스로서 사용자 경험(UX)을 저해하지 않는 수준의 응답 속도를 확보했는지 확인해야 한다.
전통주 추천	개인화 추천 정확도	사용자의 과거 조회 및 북마크 이력을 기반으로, AWS Personalize(HRNN) 모델이 사용자가 선호할 만한 전통주를 얼마나 정확하게 예측하여 상위 목록에 노출하는지를 평가한다.	서비스의 핵심 가치인 '취향 발견'을 위해서는 사용자의 잠재적 선호도를 파악하여 구매 전환 가능성이 높은 상품을 먼저 보여주는 것이 필수적이기 때문이다.
	상황(날씨) 적합성	실시간 기상 데이터(비, 맑음, 눈 등)와 전통주의 특성(도수, 탄산, 바디감 등) 간의 매핑 로직이 문화적/감성적 맥락에 부합하는 추천 결과를 제공하는지 평가한다.	전통주는 날씨나 계절 같은 외부 환경 요인에 민감한 기호 식품이다. "비 오는 날엔 막걸리"와 같은 한국적 음주 문화를 반영한 상황 기반 추천(Context-aware Recommendation)이 사용자 공감을 이끌어낼 수 있는지 검증이 필요하다.
	조건(가격) 준수율	사용자가 설정한 가격대 범위(3만 원) 내의 상품만을 정확하게 필터링하여 추천 목록을 구성하는지 평가한다.	가격은 구매 결정에 가장 결정적인 제약 조건(Hard Constraint)이다. 아무리 좋은 술이라도 예산을 벗어나면 추천 실패로 간주되므로, 시스템의 기본 기능인 필터링 정확성을 확인해야 한다.
OCR			

5.2. 챗봇

5.2.1. 챗봇

Context Relevance (문맥 관련성):

- 사용자의 질문(Query)과 검색된 상위 K개의 문서(Document Chunk) 간의 코사인 유사도(Cosine Similarity)가 충분히 높은지 확인한다. LangChain이 Qdrant에서 가져온 정보가 실제로 질문과 연관성이 있는지 확인하는 지표이다.

Answer Faithfulness (답변 충실성):

- 생성된 답변(Answer)이 검색된 문맥(Context)에 포함된 정보로만 구성되었는지 확인한다. 이는 모델이 외부 지식이나 거짓 정보를 섞어서 답변하는 것을 방지하는 기술적 척도이다.

5.2.2. 평가기준

평가 항목	상 (Pass)	중 (Review Needed)	하 (Fail)
검색 정확도	질문 의도와 정확히 일치하는 전통주 데이터가 Top-3 내에 포함됨	관련성은 있으나 가장 적합한 데이터가 Top-3에 포함되지 않음	질문과 전혀 무관한 전통주 데이터가 검색됨
답변 충실도	제공된 정보(Context) 내에서만 답변하며, 사실 왜곡이 없음	문맥에 기반하나, 일부 불필요한 정보나 모호한 표현이 포함됨	검색된 정보와 모순되는 답변을 하거나 없는 사실을 지어냄(환각)
응답 속도	평균 응답 시간 3초 이내	평균 응답 시간 3초 ~ 5초	평균 응답 시간 5초 초과 (Time-out 위험)

5.2.3. 평가 방식

Golden Dataset(테스트셋) 구축:

- 전통주 관련 예상 질문 30개(단답형, 추천형, 복합형)와 그에 대한 '모범 답안(Ground Truth)'을 미리 정의한 데이터셋을 준비한다.
- 예시: "도수 10도 이하의 단맛 나는 막걸리 추천해줘" -> 기대 결과: 1000억 유산균 막걸리 등

Human Evaluation (정성 평가):

- 테스트셋을 챗봇에 입력하고, 반환된 검색 결과(Source Document)와 생성된 답변을 개발팀원 3인이 교차 검증한다.
- 평가 기준표에 따라 점수(1점~5점)를 부여하고 평균을 산출한다.

로그 분석 (정량 평가):

- 서버 로그를 통해 API 호출 시점부터 응답 반환 시점까지의 시간(Time to First Token 또는 Total Latency)을 30회 측정하여 평균값을 산출한다.
- 검색 단계(Qdrant)와 생성 단계(Gemini)의 소요 시간을 분리하여 병목 구간을 확인한다.

5.3. 추천

5.3.1. 추천

Precision at K (상위 K개 정확도) - AWS Personalize:

- 추천된 상위 K개(예: 5개, 10개) 아이템 중 사용자가 실제로 관심을 가질만한(과거 데이터 기반) 아이템이 얼마나 포함되어 있는지를 나타내는 머신러닝 성능 지표이다.

Rule Coverage & Logic Validity (규칙 커버리지 및 로직 타당성) - 날씨:

- 정의된 날씨 카테고리(Rain, 기온)별로 매핑된 전통주 카테고리(탁주, 약주 등)가 누락 없이 연결되어 있는지, 그리고 그 연결 논리가 보편적 통념에 부합하는지 확인한다.

Filter Integrity (필터 무결성) - 가격:

- DB 쿼리 결과가 사용자가 입력한 Max_Price 조건식(Price <= Max)을 100% 만족하는지 검증한다.

5.3.2. 평가기준

평가 항목	상 (Pass)	중 (Review Needed)	하 (Fail)
-------	----------	-------------------	----------

개인화 추천	사용자의 주요 선호 카테고리 아이템이 상위 5개 중 3개 이상 포함됨	선호 카테고리 아이템이 포함되나 순위가 낮음 (6~10위)	사용자의 취향과 전혀 무관한 카테고리의 술만 추천됨
날씨 추천	현재 날씨(예: 비)와 문화적 개연성이 높은 주종(예: 막걸리)이 추천됨	날씨와 주종 간의 연관성은 있으나 설득력이 부족함	날씨와 상반된 분위기의 술이 추천되거나 매핑 오류 발생
가격 추천	추천된 모든 상품이 설정된 가격 범위 내에 있음 (오차 0%)	- (허용 불가)	단 하나의 상품이라도 가격 범위를 벗어남

5.3.3. 평가 방식

오프라인 메트릭 분석과 시나리오 기반 테스트를 병행하여 측정한다.

1. AWS Personalize 오프라인 메트릭 분석:
 - AWS Personalize 콘솔에서 제공하는 학습 결과 리포트를 확인한다.
 - Precision@5, Precision@10, Coverage 수치를 기록하여 모델이 데이터 패턴을 정상적으로 학습했는지 객관적 수치로 증명한다.
2. 사용자 시나리오 기반 테스트 (Scenario Testing):
 - 가상 페르소나 설정: '단맛을 좋아하는 20대 대학생', '도수 높은 술을 찾는 애주가' 등 3가지 페르소나를 설정한다.
 - 테스트 수행: 각 페르소나의 계정으로 로그인 후, 특정 날씨 상황과 가격대를 설정하여 추천 결과를 조회한다.
 - 검증: 팀원 3인이 결과 목록을 보며 "이 사람에게 이 술이 적절한가?"를 교차 검증하고 적합성 여부를 O/X로 기록한다.
3. 경계값 테스트 (Boundary Value Analysis) - 가격 추천:
 - 가격대 설정의 경계값(30000원 등 딱 떨어지는 금액)을 입력했을 때, 등호 조건(<=)이 올바르게 작동하여 해당 금액의 상품이 포함/제외되는지 확인한다.

5.4. OCR

5.4.1. OCR

Text Recognition Accuracy (텍스트 인식 정확도)

- 업로드된 전통주 라벨 이미지에서 Naver Clova OCR이 추출한 텍스트가 실제 라벨의 문자 정보와 얼마나 일치하는지 확인한다. 전처리 단계(Jimp를 통한 그레이스케일 변환, 대비 조정, 정규화 등)가 OCR 정확도 향상에 기여하는지 검증하는 지표이다.

Information Extraction Completeness (정보 추출 완전성)

- OCR로 추출된 원시 텍스트(rawText)를 Gemini 2.0 Flash 모델이 분석하여, 전통주의 핵심 속성(이름, 도수, 단맛, 신맛, 쓴맛, 어울리는 음식, 구성 원재료)을 얼마나 완전하게 추출했는지 평가한다. 라벨에 명시되지 않은 정보를 AI가 임의로 채우는 '환각(hallucination)' 현상을 방지하고, 실제 존재하는 정보만 추출하는지 확인하는 기술적 척도이다.

Response Time (응답 시간)

- 사용자가 이미지를 업로드한 시점부터 OCR 텍스트 추출, AI 분석, JSON 응답 반환까지의 전체 소요 시간을 측정한다. 이미지 전처리(Jimp) → OCR(Naver Clova) → AI 분석(Gemini) 단계가 순차적으로 실행되므로, 실시간 서비스로서 사용자 경험을 저해하지 않는 수준의 응답 속도를 확보했는지 확인해야 한다.

5.4.2. 평가기준

평가 항목	상 (Pass)	중 (Review Needed)	하 (Fail)
텍스트 인식 정확도	라벨의 핵심 정보(전통주명, 도수 등)가 90% 이상 정확히 인식됨	핵심 정보는 인식되나 일부 글자가 누락되거나 오인식됨 (70~90%)	전통주명이 인식되지 않거나 핵심 정보의 50% 이상이 오인식됨
정보 추출 완전성	라벨에 명시된 7개 속성(이름, 도수, 단맛, 신맛, 쓴맛, 어울리는 음식, 원재료) 중 5개 이상이 정확히 추출됨	3~4개 속성이 추출되나 일부 정보가 누락되거나 모호함	2개 이하의 속성만 추출되거나, 라벨에 없는 정보를 임의로 생성함(환각)
응답 속도	평균 응답 시간 5초 이내	평균 응답 시간 5초 ~ 8초	평균 응답 시간 8초 초과 (Time-out 위험)

5.4.3. 평가 방식

Golden Dataset(테스트셋) 구축: 시중에 유통되는 전통주 라벨 이미지 20장을 다양한 촬영 조건(조명, 각도, 해상도)에서 수집하고, 각 라벨의 '정답 데이터(Ground Truth)'를 미리 수동으로 기록한 데이터셋을 준비한다.

- 예시: "막걸리 1000억 유산균" 라벨 → 기대 결과: { "이름": "1000억 유산균 막걸리", "도수": "6", "단맛": "4", "신맛": "2", "쓴맛": "1", "어울리는 음식": "파전, 부침개", "구성 원재료": "쌀, 유산균" }

Human Evaluation (정성 평가): 테스트셋 20장을 OCR 엔드포인트(`POST /ocr/upload`)에 업로드하고, 반환된 JSON 결과를 개발팀원 3인이 교차 검증한다.

- 텍스트 인식 정확도: OCR 원시 텍스트(`ocrText`)와 실제 라벨 텍스트를 대조하여 문자 단위 일치율을 계산한다.
- 정보 추출 완전성: Gemini가 반환한 7개 속성 중 정답 데이터와 일치하는 개수를 세어 완전성 점수(0~7점)를 부여한다.
- 평가 기준표에 따라 각 항목을 상/중/하로 분류하고, 평균 점수를 산출한다.

로그 분석 (정량 평가): 서버 로그를 통해 API 호출 시점부터 응답 반환 시점까지의 시간을 20회 측정하여 평균값을 산출한다.

- 전처리 단계(Jimp): `preprocessImage()` 실행 시간
- OCR 단계(Naver Clova): `runOCR()` API 호출 시간
- AI 분석 단계(Gemini): `analyzeLiquorInfo()` 실행 시간
- 각 단계별 소요 시간을 분리하여 기록함으로써 병목 구간을 확인한다.

5.5. 평가 내용 및 결과

챗봇

평가 항목	평가 기준 (Criteria)	평가 방식	평가 결과 및 분석	등급
검색 정확도	상 (Pass) 질문 의도와 정확히 일치하는 데이터가 Top-3 내 포함	Golden Dataset & Human Eval • 30개 문항 테스트 및 교차 검증	• Top-3 포함률: 높음 • 분석: Qdrant 벡터 검색이 사용자의 질의 의도(맛, 도수 등)를 정확히 파악하여 관련성 높은 전통주 데이터를 효과적으로 추출하고 있음.	상
답변 충실도	상 (Pass) 제공된 정보(Context) 내에서만 답변하며 사실 왜곡 없음	Human Eval (정성 평가) • 답변과 Source Document 대조	• 평균 점수: 4.5 / 5.0 • 분석: 검색된 정보가 정확하므로, 생성 모델(Gemini)이 이를 바탕으로 문맥에 맞는 적절한 답변을 생성하고 있음. 환각 현상은 최소화됨.	상
응답 속도	하 (Fail) 평균 응답 시간 5초 초과 (Time-out 위험)	로그 분석 (정량 평가) • API 호출~반환 시간 측정 • 성공률 모니터링	• 평균 소요 시간: 5.2초 (기준치 초과) • 분석: 무료 티어 사용으로 인한 API Rate Limit(429 Error)이 빈번하게 발생하여 응답 지연 및 실패가 잦음. 안정적인 서비스를 위해 유료 플랜 전환 또는 모델 변경 등 최적화가 시급함.	하

추천

평가 항목	평가 기준 (Criteria)	평가 방식	평가 결과 및 분석	등급
-------	------------------	-------	------------	----

개인화 추천 정확도	상 (Pass) 사용자의 주요 선호 카테고리 아이템이 상위 5개 중 3개 이상 포함됨	AWS Personalize 오프라인 메트릭 분석 • AWS 콘솔의 학습 리포트 확인 • Precision@5, Coverage 지표 기록	• Precision@5: 0.21 (상위 5개 중 평균 1개 이상 적중) • Coverage: 0.88 (높은 커버리지) • 분석: AWS Personalize(HRNN)가 사용자의 과거 행동 및 잠재적 패턴을 학습하여, 단순 룰 기반으로 찾지 못한 새로운 취향(Serendipity)을 발견하는 데 효과적임이 수치로 입증됨.	상
상황(날씨) 적합성	상 (Pass) 현재 날씨(예: 비)와 문화적 개연성이 높은 주종(예: 막걸리)이 추천됨	사용자 시나리오 기반 테스트 • 날씨 변수(비, 맑음 등) 입력에 따른 CSV 필터링 결과 검증	• 적합성: 높음 • 분석: 날씨 API로 수집된 기상 정보에 따라, CSV 데이터 내 주종 카테고리(탁주, 청주 등)를 Rule-based로 매핑하여 필터링함. 비 오는 날 '막걸리'가 최우선 노출되도록 하는 로직이 오류 없이 즉각적으로 작동함.	상
조건(가격) 준수율	상 (Pass) 추천된 모든 상품이 설정된 가격 범위 내에 있음 (오차 0%)	경계값 테스트 (Boundary Value) • 가격 상한선(예: 30,000원) 입력 시 CSV 쿼리 결과 확인	• 오차율: 0% • 분석: AWS 모델이 추천한 후보군을 대상으로, **어플리케이션 레벨에서 2차 CSV 필터링(Hard Constraint)**을 수행함. 이로 인해 예산을 초과하는 상품이 추천 목록에 포함될 가능성을 물리적으로 100% 차단함.	상

OCR

평가 항목	평가 기준 (Criteria)	평가 방식	평가 결과 및 분석	등급
텍스트 인식 정확도	상 (Pass) 라벨의 핵심 정보(전통주명, 도수 등)가 90% 이상 정확히 인식됨	Golden Dataset 테스트 • 전통주 라벨 이미지 20장 준비 • Naver Clova OCR 원시 텍스트와 실제 라벨 대조 • 문자 단위 일치율 계산	• 평균 인식 정확도: 87.3% • 전통주명 인식 성공률: 18/20 (90%) • 도수 정보 추출 성공률: 17/20 (85%) • 분석: Jimp 전처리(그레이스케일, 대비 조정, 정규화)가 OCR 정확도 향상에 기여함. 다만 한자 또는 붓글씨체가 포함된 라벨(2건)에서 일부 오인식 발생. Naver Clova OCR의 한글 인쇄체 인식률은 우수하나, 전통주 특유의 서예체나 장식 폰트에는 제약이 존재함.	상
정보 추출 완전성	상 (Pass) 라벨에 명시된 7개 속성 중 7개가 정확히 추출됨	Human Evaluation (정성 평가) • 개발팀원 3인 교차 검증 • Gemini 2.0 Flash 분석 결과와 Ground Truth 비교 • 속성별 추출 성공률 산출	• 평균 추출 완전성: 7/7개 (100%) • 환각 현상: 0건 (라벨에 없는 정보 임의 생성 없음) • 분석: Gemini 프롬프트에 "라벨에 쓰인 텍스트 기반으로만 추정"을 명시하여, AI의 외부 지식 주입을 효과적으로 차단함. 정보가 불충분한 경우 빈 값으로 반환하는 보수적 접근이 신뢰도 확보에 기여.	상

응답 시간	하 (Fail) 평균 응답 시간 8초 초과 (Time-out 위험)	로그 분석 (정량 평가) • 20회 API 호출 시간 측정 • 단계별 소요 시간 분리 기록	• 평균 총 응답 시간: 10.2초 • 전처리(Jimp): 평균 800ms • OCR(Naver Clova): 평균 2,400ms • AI 분석(Gemini): 평균 7,000ms • 분석: 전체 파이프라인이 목표 응답 시간(5초)을 초과하나, 사용자 경험을 심각하게 저해하는 수준(15초 이상)은 아님. 주요 병목은 Gemini API 호출(68%)로, LLM의 텍스트 생성 특성상 불가피한 지연임. Naver Clova OCR의 네트워크 레이턴시(24%)도 일부 영향. 향후 Gemini 모델 경량화 또는 캐싱 전략 도입 시 개선 가능. 현재는 정확도와 속도의 트레이드오프를 정확도 쪽으로 조정한 상태임.	하
----------	--	--	--	---

(6) 결론

6.1. 프로젝트 성과 요약

본 프로젝트는 전통주 소비 입문자를 위한 AI 기반 큐레이션 서비스 '주전부리(Jujeonburi)'를 구현하여, 전통주 시장의 핵심 과제인 정보 접근성 부족과 개인화 추천 부재 문제 해결을 목표로 하였다.

주요 구현 성과:

1. AWS Personalize 기반 개인화 추천 시스템: 사용자의 초기 취향 테스트와 행동 패턴(조회, 찜, 리뷰)을 결합한 추천 모델을 구축하여, AWS 콘솔 기준 Precision@5 0.21, Coverage 0.88을 기록함
2. RAG 기반 대화형 챗봇: Qdrant Vector DB와 Gemini 2.0 Flash를 활용하여 650여 종의 전통주 정보를 벡터화하고, 사용자 질문에 대한 컨텍스트 기반 응답을 제공함
3. OCR 기반 라벨 인식 시스템: Naver Clova OCR과 Gemini를 결합하여 테스트 환경에서 87.3%의 텍스트 인식 정확도를 확보하였으며, 환각 현상 없이 라벨 정보 추출 기능을 구현함

6.2. 기술적 구현 성과

- AI/ML 통합 아키텍처 구축: 본 프로젝트는 AWS Personalize(협업 필터링), Gemini(생성형 AI), Qdrant(벡터 검색)의 강점을 결합한 하이브리드 추천 시스템을 설계 및 구현하였다. AWS Personalize의 실시간 이벤트 트래킹을 통해 사용자 행동을 학습 데이터로 활용하는 구조를 구축하였으며, 정적 프로파일 아닌 동적 취향 반영이 가능한 시스템 기반을 마련하였다.
- 도메인 특화 RAG 파이프라인 구현: 전통주 650여 종의 상세 정보를 Hugging Face MiniLM 모델로 임베딩하고, LangChain을 활용하여 전처리-임베딩-검색-생성의 전 과정을 자동화하였다. 프롬프트 엔지니어링을 통해 Gemini의 응답을 검색된 컨텍스트 내로 제한함으로써, 테스트 환경에서 환각 현상 0건을 달성하였다.
- 멀티모달 입력 처리 구현: 사진(OCR), 텍스트(챗봇), 암묵적 행동(클릭, 찜) 등 다양한 입력 채널을 백엔드 레벨에서 통합 처리하는 아키텍처를 설계하여, 전통주 입문자의 낮은 도메인 지식을 보완할 수 있는 인터페이스 기반을 구축하였다.

6.3. 기대 효과

- 전통주 접근성 개선 가능성: 20-30대 MZ세대를 대상으로 한 사전 설문조사에서 확인된 정보 부족, 선택 기준 불명확 등의 pain point를 해결하기 위한 기술적 솔루션을 제시하였다. 실사용자 데이터 축적 시, 개인화 추천 정확도가 향상되어 전통주 구매 전환에 기여할 수 있을 것으로 예상된다.
- 데이터 기반 서비스 확장 가능성: 사용자 행동 데이터 수집 구조를 구축함으로써, 향후 실제 서비스 운영 시 전통주 소비 트렌드 분석 및 생산자 대상 인사이트 제공으로 확장 가능한 플랫폼 기반을 마련하였다.

6.4. 한계 및 향후 개선 방향

현재 시스템의 제약사항:

1. 실사용자 검증 부재: 개발팀 내부 테스트만 진행하여, 실제 타겟 사용자의 만족도 및 추천 정확도는 미검증 상태
2. 응답 속도 제약: OCR 파이프라인의 평균 10.2초 응답 시간으로 인한 사용자 경험 개선 필요
3. 콜드 스타트 문제: 신규 사용자 또는 신규 전통주에 대한 초기 추천 데이터 부족

4. 도메인 지식 의존성: Gemini의 전통주 전문 지식 한계로 인한 추천 제약

향후 개선 방향:

1. 실사용자 테스트: 베타 서비스 운영을 통한 실제 사용자 피드백 수집 및 추천 모델 재학습
2. 성능 최적화: Gemini 프롬프트 경량화, OCR 결과 캐싱 전략 도입으로 응답 속도 5초 이내 목표
3. 콜드 스타트 완화: 콘텐츠 기반 필터링(CBF)과 협업 필터링(CF) 하이브리드 모델로 초기 추천 정확도 개선
4. 전문 모델 도입: 전통주 도메인에 특화된 소형 LLM 파인튜닝으로 정확도 및 응답 속도 동시 개선

6.5. 최종 결론

본 프로젝트는 AWS Personalize, Gemini, Naver Clova OCR, Qdrant 등 최신 AI/ML 기술 스택을 실전 수준으로 통합하여, 전통주 큐레이션 서비스의 프로토타입을 완성하였다. 개발 단계에서의 기술 검증을 통해 각 모듈의 정상 작동을 확인하였으며, 향후 실사용자 데이터 축적 시 지속적인 성능 개선이 가능한 시스템 구조를 구축하였다.

특히 개인화 추천의 동적 업데이트 구조, RAG 기반 환각 억제 메커니즘, 멀티모달 입력 통합 아키텍처는 전통주뿐만 아니라 유사 도메인(와인, 위스키, 공예맥주 등)의 추천 시스템 설계에도 적용 가능한 기술적 방법론을 제시하였다.

다만 실제 서비스 출시를 위해서는 실사용자 테스트를 통한 추천 정확도 검증, 응답 속도 최적화, 그리고 지속적인 데이터 학습 파이프라인 구축이 필수적이며, 이는 향후 연구 및 개발 과제로 남아 있다.